

claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\workflows\wf_3e301d0b-e59\agent-a9d2d00cbd03f1c7d.jsonl`
- SHA-256: `3d8528711fd98339f0aa9567262e6141a588a6067ea949b6f823d2f503362a81`
- Source modified: `2026-06-13T18:46:00+00:00`
- Imported at: `2026-07-05T16:48:25+00:00`
- project: `wf_3e301d0b-e59`
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`
```

Transcript

1. user (2026-06-13T18:45:07.540Z)

Map the typed config so we can add knobs safely.
Read: backend/config/ (models.py IndexingConfig + any nested configs like WasbConfig/TrackNetConfig), and how config.json + the unknown-key warning + extra= behave.
Report: how IndexingConfig is structured and validated (pydantic), how nested sub-configs are declared, where default_segmenter lives, and the EXACT additions to make: (1) indexing.min_crossings: int = 0; (2) an optional indexing.fusion sub-config with fields court_polygon (optional list of [x,y] normalized 0-1 points; None=no mask), net_axis ('x'|'y') + net_position (normalized), expected_players (int). Note any extra='forbid'/alias gotchas. Repo root = C:/Users/avidu/Projects/badminton-highlight-indexer (the indexer). Read the ACTUAL files and quote file:line. Be precise, not generic.

2. assistant (2026-06-13T18:45:12.134Z)

I'll map the config structure for you. Let me start by exploring the backend config directory and understanding the current Pydantic setup.

3. user (2026-06-13T18:45:12.501Z)

```
backend\config\loader.py
backend\config\__init__.py
backend\config\models.py
```

4. user (2026-06-13T18:45:12.573Z)

```
EISDIR: illegal operation on a directory, read 'C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config'
```

5. assistant (2026-06-13T18:45:13.944Z)

Now let me read the models.py file:

6. user (2026-06-13T18:45:14.587Z)

```
1      """Pydantic models for the application configuration.
2
3      This module defines the canonical, validated, typed configuration for the
4      Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6      The models support graceful migration from old config.json files via extra="allow".
7      """
8
9      from __future__ import annotations
10
```

```

11     from typing import Any, List, Literal, Optional
12
13     from pydantic import BaseModel, Field, field_validator, model_validator
14
15     Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
18     class ValidationRelevanceConfig(BaseModel):
19         court_green_lower: list[int] = Field(default=[35, 40, 40])
20         court_green_upper: list[int] = Field(default=[85, 255, 255])
21         court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23
24     class ValidationConfig(BaseModel):
25         min_resolution_width: int = 1280
26         min_resolution_height: int = 720
27         min_fps: float = 29.9
28         min_blur_threshold: float = 20.0
29         max_scene_cuts_per_minute: float = 0.5
30         relevance: ValidationRelevanceConfig =
31         Field(default_factory=ValidationRelevanceConfig)
32
33     class TrackNetConfig(BaseModel):
34         wsl_distro: str = "Ubuntu"
35         repo_dir: str = "~/models/TrackNetV4"
36         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
37         conda_env: str = "TrackNetV4"
38         weights_path: str = ""
39         queue_length: int = 5
40         stage_in_wsl: bool = True
41         wsl_stage_dir: str = "~/clips"
42         timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
43         velocity_threshold: float = 0.02
44         # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
45         # decoded frame cache) are deleted automatically ? they are pure intermediates,
46         # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
47         # true only for debugging. On FAILURE they are always kept so a re-run can resume.
48         keep_frames: bool = False
49         # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB.
50         # 0 = disabled.
51         min_free_gb: float = 0.0
52
53
54     class WasbIndexConfig(BaseModel):
55         """Typed config for the live WASB shuttle substrate (indexing.wasb).
56
57         Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
58         these knobs actually take effect ? previously this whole block was silently dropped
59         because nested config sections defaulted to extra='ignore'.
60         """
61         wsl_distro: str = "Ubuntu"
62         repo_dir: str = "~/models/WASB-SBDT"
63         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
64         conda_env: str = "wasb"
65         weights_path: str = "~/models/WASB-
66         SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
67         sport: str = "badminton"
68         wsl_stage_dir: str = "~/clips"
69         timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
70         velocity_threshold: float = 0.02
71         # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
72         # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
73         # regenerable
74         # from the source. Set true only for debugging. On FAILURE they are kept so a re-run

```

```

resumes.
73         keep_frames: bool = False
74         # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB. 0 =
disabled.
75         min_free_gb: float = 0.0
76
77
78     class IndexingConfig(BaseModel):
79         default_segmenter: str = "yolo_hybrid"
80         use_gpu: bool = True
81         motion_threshold: float = 0.08
82         min_segment_duration: float = 3.0
83         dead_time_merge_gap: float = 2.0
84         chunk_duration: float = 90.0
85         chunk_overlap: float = 10.0
86         detector_model: str = "fasterrcnn_resnet50_fpn"
87         detector_score_threshold: float = 0.5
88         yolo_velocity_threshold: float = 0.15
89         yolo_frame_skip: int = 3
90         yolo_tracker: str = "bytetrack.yaml"
91         yolo_prefetch_workers: int = 2
92         min_action_window_duration: float = 2.0
93         log_yolo_candidates: bool = True
94         store_yolo_candidates: bool = True
95         ai_handoff_padding: float = 2.0
96         ai_max_workers: int = 5
97         # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
98         # chunks of a high-bitrate (4K) source blow past the provider upload cap
99         # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
100        # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
101        # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
102        # Matches gemini_refine's --clip-scale-width default.
103        ai_chunk_scale_width: int = 1280
104        # Optional debug knob: trim every video to the first N seconds before processing.
105        debug_duration: Optional[float] = None
106
107        tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
108        wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
109
110        @field_validator("default_segmenter")
111        @classmethod
112        def segmenter_must_be_registered(cls, v: str) -> str:
113            # Registry check happens at runtime in main/segmenter selection.
114            # Here we just allow any string for forward compatibility.
115            return v
116
117        @model_validator(mode="after")
118        def _chunk_step_must_be_positive(self) -> "IndexingConfig":
119            # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
positive
120            # step makes `while t < duration: t += step` loop forever, growing memory before
any
121            # GPU work. Reject the bad config at load time instead.
122            if self.chunk_overlap >= self.chunk_duration:
123                raise ValueError(
124                    f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
"
125                    f"({self.chunk_duration}); otherwise chunking never advances."
126                )
127            return self
128
129
130     class OutputConfig(BaseModel):
131         default_highlights_name: str = "highlights.mp4"
132         db_name: str = "sports_indexer.db"

```

```

133     # Directory where the DB, highlight reels, and processed clips are written.
134     # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
135     output_dir: str = "output"
136
137
138     class WatermarkConfig(BaseModel):
139         """Branding overlay burned into each highlight clip during the per-clip re-encode
140         (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
141         (under `stitching.watermark` in config.json) to turn it off. The text is fully
142         configurable. The concat stage stays stream-copy (-c copy)."""
143         enabled: bool = True
144         text: str = "Generated by Khelsutra.guru"
145         logo_path: str = "" # optional transparent PNG overlay; "" = no logo
146         font_path: str = "" # optional .ttf; "" = use the fontconfig `font` family
147         font: str = "Sans" # fontconfig family used when font_path is empty
148         font_size_ratio: float = Field(default=0.035, gt=0.0, le=0.5) # text height as a
149         fraction of frame height
150         opacity: float = Field(default=0.85, ge=0.0, le=1.0)
151         box: bool = True # faint backing box behind the text for legibility
152         margin: int = Field(default=24, ge=0) # px inset from the chosen corner
153         position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = "bottom-
154         right"
155
156     class StitchingConfig(BaseModel):
157         begin_padding: float = 0.5
158         end_padding: float = 0.5
159         use_cache: bool = False
160         min_shot_count: int = 1
161         watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)
162
163     class SecurityConfig(BaseModel):
164         # When set, /api/process video_path must resolve under this directory (hosted/tenant
165         # mode). Default None preserves the single-user local 'any local file' workflow.
166         allowed_video_root: Optional[str] = None
167
168         # --- Chunked upload (B2, PR-2) ---
169         # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
170         # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
171         # contain upload_dir or /api/process will reject the uploaded paths.
172         upload_dir: str = "output/uploads"
173         # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
174         # free-tier edge proxies cap request bodies there (HOSTING_PLAN $2).
175         max_upload_mb: int = 4096
176         max_part_mb: int = 95
177         allowed_upload_extensions: List[str] = ["mp4", "mov"]
178
179
180     class AppConfig(BaseModel):
181         """Root configuration object.
182
183         All runtime configuration should be accessed via an instance of this class
184         (or the loader) rather than raw dict lookups.
185         """
186
187         sport: Sport = "badminton"
188         ai_provider: str = "gemini"
189         # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
190         # the alias never deprecates underneath us. Eval reports record the run date
191         # alongside.
192         ai_model: str = "gemini-flash-latest"

```

```

193 validation: ValidationConfig = Field(default_factory=ValidationConfig)
194 indexing: IndexingConfig = Field(default_factory=IndexingConfig)
195 output: OutputConfig = Field(default_factory=OutputConfig)
196 stitching: StitchingConfig = Field(default_factory=StitchingConfig)
197 security: SecurityConfig = Field(default_factory=SecurityConfig)
198
199 # Allow extra keys during transition so existing config.json files do not break.
200 model_config = {
201     "extra": "allow",
202     "validate_assignment": True,
203 }
204
205 # Temporary migration shim for legacy dict-style access (config.get("section",
206 {}).get(...)).
207 # Will be removed once all call sites (validators, segmenters, etc.) move to typed
208 # attribute access / the typed config. A nested *section* is returned as a plain
dict
209 # via model_dump, so legacy chained access keeps working ? returning the Pydantic
210 # section model instead breaks it (a model has no .get), which bit every validator
on
211 # real runs. Scalar leaves are returned as-is.
212 def get(self, key: str, default: Any = None) -> Any:
213     _MISSING = object()
214     val: Any = _MISSING
215     if key in type(self).model_fields or hasattr(self, key):
216         val = getattr(self, key, _MISSING)
217     if val is _MISSING:
218         # Legacy convenience: find a leaf key inside any nested section. Sections
are
219         # discovered dynamically from the model fields, so a future top-level
section
220         # (e.g. `audio`, `jobs`) is covered automatically without editing a
hardcoded list.
221         for fname in type(self).model_fields:
222             section = getattr(self, fname, None)
223             if isinstance(section, BaseModel) and hasattr(section, key):
224                 val = getattr(section, key)
225                 break
226         if val is _MISSING:
227             return default
228         if isinstance(val, BaseModel):
229             return val.model_dump(mode="json")
230         return val
231
232 def __getitem__(self, key: str) -> Any:
233     val = self.get(key)
234     if val is None and key not in type(self).model_fields:
235         # Re-raise like a real dict for unknown top-level
236         raise KeyError(key)
237     return val
238
239 # Convenience type alias for callers that want to annotate
240 Config = AppConfig
241

```

7. user (2026-06-13T18:45:14.820Z)

```

1     """Configuration loader and saver.
2
3     Provides a single place to load validated AppConfig with built-in defaults
4     merged with optional file overrides (config.json).
5
6     This replaces scattered json.load + .get() patterns throughout the codebase.
7     """

```

```

8
9     from __future__ import annotations
10
11     import json
12     import logging
13     from pathlib import Path
14     from typing import Any
15
16     from pydantic import BaseModel
17
18     from .models import AppConfig
19
20     logger = logging.getLogger(__name__)
21
22     DEFAULT_CONFIG_PATH = Path("config.json")
23
24
25     def _unknown_key_paths(data: dict[str, Any], model_cls: type[BaseModel], prefix: str =
26     "") -> list[str]:
27         """Dotted paths in `data` that the typed config will not honor.
28
29         Two silent failure modes exist without this check: a typo'd TOP-LEVEL key is
30         kept by AppConfig's ``extra="allow"`` but never read by typed code, and a
31         typo'd key INSIDE a section is silently dropped by Pydantic. Either way the
32         user's intent is lost ? collect every such key so load_config can warn.
33         """
34         unknown: list[str] = []
35         fields = model_cls.model_fields
36         for key, val in data.items():
37             if key not in fields:
38                 unknown.append(prefix + key)
39                 continue
40             ann = fields[key].annotation
41             if isinstance(val, dict) and isinstance(ann, type) and issubclass(ann,
42             BaseModel):
43                 unknown.extend(_unknown_key_paths(val, ann, f"{prefix}{key}.")
44             return unknown
45
46     def _get_builtin_defaults() -> dict[str, Any]:
47         """Return a plain dict of the current built-in defaults.
48
49         The AppConfig model is the single source of truth for defaults.
50         """
51         return AppConfig().model_dump(mode="json")
52
53     def load_config(path: str | Path | None = None) -> AppConfig:
54         """Load configuration with the following precedence:
55
56         1. Built-in defaults defined in the Pydantic models.
57         2. Values from the JSON file (if present and readable).
58         3. (Future) environment / CLI overrides.
59
60         Missing file or unreadable file ? returns a fully defaulted AppConfig.
61         Unknown keys in the file are tolerated (extra="allow" on the model).
62         """
63         cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH
64
65         file_data: dict[str, Any] = {}
66         if cfg_path.exists():
67             try:
68                 with open(cfg_path, "r", encoding="utf-8") as f:
69                     file_data = json.load(f)
70             except Exception as exc:

```

```

71         # Non-fatal: continue with defaults + whatever we could parse.
72         # In production we might log here.
73         logger.warning(f"Failed to read {cfg_path}: {exc}. Using defaults + partial
data.")
74
75     if file_data:
76         unknown = _unknown_key_paths(file_data, AppConfig)
77         if unknown:
78             logger.warning(
79                 "[CONFIG WARNING] %s contains keys the typed config does not "
80                 "recognize (typo'd or removed knobs ? top-level extras are kept "
81                 "but unread; unknown section keys are silently dropped): %s",
82                 cfg_path, ", ".join(sorted(unknown)),
83             )
84
85     # Merge: start from defaults, overlay file data (Pydantic will validate/coerce).
86     merged = {**_get_builtin_defaults(), **file_data}
87
88     return AppConfig.model_validate(merged)
89
90
91 def save_default_config(path: str | Path | None = None) -> Path:
92     """Write a fresh default config.json.
93
94     Used by setup.py diagnostics and the --setup CLI path.
95     Overwrites if the file exists.
96     """
97     cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH
98     cfg = AppConfig()
99     cfg_path.parent.mkdir(parents=True, exist_ok=True)
100    cfg_path.write_text(cfg.model_dump_json(indent=4), encoding="utf-8")
101    return cfg_path
102
103
104 # Backwards-compatible alias used by some older call sites during transition.
105 def get_default_config() -> dict[str, Any]:
106     """Return defaults as a plain dict (for transitional use only)."""
107     return _get_builtin_defaults()
108

```

8. user (2026-06-13T18:45:14.968Z)

```

1     """Configuration package.
2
3     Public API:
4         from backend.config import load_config, AppConfig, Config
5
6     This package is the canonical place for all configuration access.
7     """
8
9     from .loader import load_config, save_default_config
10    from .models import AppConfig, Config, IndexingConfig, StitchingConfig, ValidationConfig
11    # re-export common ones
12
13    __all__ = [
14        "load_config",
15        "save_default_config",
16        "AppConfig",
17        "Config",
18        "IndexingConfig",
19        "StitchingConfig",
20        "ValidationConfig",
21    ]

```

9. assistant (2026-06-13T18:45:16.712Z)

Perfect. Now let me search for any usage of these configs and verify how the config.json structure is consumed:

10. user (2026-06-13T18:45:17.478Z)

C:\Users\avidu\Projects\badminton-highlight-indexer/config.json

11. user (2026-06-13T18:45:17.916Z)

Found 15 files

```
backend\pipeline\detectors\person_detector.py
tools\cleanup_caches.py
backend\pipeline\detectors\wasb_runner.py
backend\pipeline\detectors\tracknet_runner.py
backend\config\models.py
backend\main.py
scratch\run_gx010125.py
run_process_and_stitch.py
backend\pipeline\segmenters\trajectory_hybrid.py
tests\test_config.py
backend\pipeline\segmenters\gemini.py
backend\providers\gemini.py
backend\reporting\harvest.py
tests\test_config_wasb.py
tests\test_reporting_harvest.py
```

12. assistant (2026-06-13T18:45:19.172Z)

Let me check the actual config.json and look at a few key files to see how nested config is accessed:

13. user (2026-06-13T18:45:19.422Z)

```
1      {
2          "sport": "badminton",
3          "ai_provider": "gemini",
4          "ai_model": "gemini-flash-latest",
5          "validation": {
6              "min_resolution_width": 1280,
7              "min_resolution_height": 720,
8              "min_fps": 29.9,
9              "min_blur_threshold": 20.0,
10             "max_scene_cuts_per_minute": 0.5,
11             "relevance": {
12                 "court_green_lower": [
13                     35,
14                     40,
15                     40
16                 ],
17                 "court_green_upper": [
18                     85,
19                     255,
20                     255
21                 ],
22                 "court_pixels_min_ratio": 0.05
23             }
24         },
25         "indexing": {
26             "default_segmenter": "gemini",
27             "use_gpu": true,
28             "motion_threshold": 0.08,
29             "min_segment_duration": 3.0,
```



```

30         "dead_time_merge_gap": 2.0,
31         "chunk_duration": 90,
32         "chunk_overlap": 10,
33         "detector_model": "fasterrcnn_resnet50_fpn",
34         "detector_score_threshold": 0.5,
35         "yolo_velocity_threshold": 0.15,
36         "yolo_frame_skip": 3,
37         "yolo_tracker": "bytetrack.yaml",
38         "yolo_prefetch_workers": 2,
39         "min_action_window_duration": 2.0,
40         "log_yolo_candidates": true,
41         "store_yolo_candidates": true,
42         "ai_handoff_padding": 2.0,
43         "ai_max_workers": 1,
44         "tracknet": {
45             "wsl_distro": "Ubuntu",
46             "repo_dir": "~/models/TrackNetV4",
47             "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
48             "conda_env": "TrackNetV4",
49             "weights_path": "",
50             "queue_length": 5,
51             "stage_in_wsl": true,
52             "wsl_stage_dir": "~/clips",
53             "timeout_sec": 1800,
54             "velocity_threshold": 0.02
55         },
56         "wasb": {
57             "wsl_distro": "Ubuntu",
58             "repo_dir": "~/models/WASB-SBDT",
59             "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
60             "conda_env": "wasb",
61             "weights_path": "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar",
62             "sport": "badminton",
63             "wsl_stage_dir": "~/clips",
64             "timeout_sec": 1800,
65             "velocity_threshold": 0.02
66         }
67     },
68     "output": {
69         "default_highlights_name": "highlights.mp4",
70         "db_name": "sports_indexer.db"
71     },
72     "stitching": {
73         "begin_padding": 0.5,
74         "end_padding": 0.5,
75         "use_cache": false,
76         "min_shot_count": 1,
77         "watermark": {
78             "enabled": true,
79             "text": "Generated by Khelsutra.guru",
80             "position": "bottom-right",
81             "opacity": 0.85,
82             "logo_path": "",
83             "font_path": ""
84         }
85     }
86 }

```

14. user (2026-06-13T18:45:19.629Z)

```

1     """Basic tests for the typed configuration package.
2
3     These ensure:
4     - Defaults are sensible and come from the models.

```

```

5     - File overrides work.
6     - Old config files with extra keys are tolerated.
7     - save_default_config produces loadable output.
8     """
9
10    import json
11    import tempfile
12    import os
13
14
15    from backend.config import load_config, save_default_config, AppConfig
16
17
18    def test_load_default_config():
19        # Force load without the project's config.json so we test the model built-in
defaults
20        cfg = load_config("/nonexistent/path/that/does/not/exist/12345/config.json")
21        assert isinstance(cfg, AppConfig)
22        assert cfg.sport == "badminton"
23        assert cfg.indexing.default_segmenter == "yolo_hybrid"
24        assert cfg.validation.min_blur_threshold == 20.0 # updated default from review
25        assert cfg.indexing.ai_max_workers == 5
26
27
28    def test_file_override():
29        with tempfile.TemporaryDirectory() as td:
30            p = os.path.join(td, "override.json")
31            data = {
32                "indexing": {
33                    "default_segmenter": "gemini",
34                    "ai_max_workers": 3,
35                },
36                "validation": {
37                    "min_blur_threshold": 42.0,
38                },
39            }
40            with open(p, "w") as f:
41                json.dump(data, f)
42
43            cfg = load_config(p)
44            assert cfg.indexing.default_segmenter == "gemini"
45            assert cfg.indexing.ai_max_workers == 3
46            assert cfg.validation.min_blur_threshold == 42.0
47            # Unspecified keys fall back to defaults
48            assert cfg.sport == "badminton"
49
50
51    def test_extra_keys_tolerated():
52        """Old or future keys in config.json must not break loading."""
53        with tempfile.TemporaryDirectory() as td:
54            p = os.path.join(td, "with_extra.json")
55            data = {
56                "sport": "badminton",
57                "indexing": {"default_segmenter": "yolo_hybrid"},
58                "some_future_key": "ignored_for_now",
59                "validation": {"relevance": {"court_pixels_min_ratio": 0.1}},
60            }
61            with open(p, "w") as f:
62                json.dump(data, f)
63
64            cfg = load_config(p)
65            assert cfg.indexing.default_segmenter == "yolo_hybrid"
66            assert cfg.validation.relevance.court_pixels_min_ratio == 0.1
67
68

```

```

69 def test_unknown_keys_warn_with_dotted_paths(tmp_path, caplog):
70     """Typo'd keys must be LOUD: top-level extras are kept-but-unread and unknown
71     section keys are silently dropped, so without this warning a typo just makes
72     the knob a no-op (the audit's 'silently dropped config' class)."""
73     p = tmp_path / "typo.json"
74     p.write_text(json.dumps({
75         "some_future_key": 1,                                # top-level extra
76         "indexing": {"velocity_threshold": 0.5},             # typo'd section key
77         "validation": {"relevance": {"unknown_gate": True}}, # doubly nested
78     })))
79     import logging
80     with caplog.at_level(logging.WARNING, logger="backend.config.loader"):
81         cfg = load_config(str(p))
82         assert isinstance(cfg, AppConfig) # still non-fatal
83         joined = " ".join(r.message for r in caplog.records)
84         assert "some_future_key" in joined
85         assert "indexing.velocity_threshold" in joined
86         assert "validation.relevance.unknown_gate" in joined
87
88
89 def test_recognized_config_does_not_warn(tmp_path, caplog):
90     p = tmp_path / "clean.json"
91     p.write_text(json.dumps({
92         "sport": "badminton",
93         "indexing": {"default_segmenter": "gemini", "wasb": {"velocity_threshold":
0.02}},
94     })))
95     import logging
96     with caplog.at_level(logging.WARNING, logger="backend.config.loader"):
97         load_config(str(p))
98     assert not [r for r in caplog.records if "CONFIG WARNING" in r.message]
99
100
101 def test_save_default_config_roundtrip():
102     with tempfile.TemporaryDirectory() as td:
103         p = os.path.join(td, "saved.json")
104         saved = save_default_config(p)
105         assert str(saved) == p
106         assert os.path.exists(p)
107
108         cfg = load_config(p)
109         assert cfg.indexing.default_segmenter == "yolo_hybrid"
110         assert cfg.validation.min_blur_threshold == 20.0
111
112
113 def test_model_is_the_source_of_truth():
114     """Changing a default in the model should be reflected without touching json."""
115     # This is a documentation test more than anything.
116     cfg = AppConfig()
117     assert cfg.indexing.tracknet.velocity_threshold == 0.02
118
119
120 def test_output_dir_field_default_and_override():
121     """output_dir now lives on OutputConfig (was a write-only runtime hack), so
122     /api/compile and the CLI resolve the same value. Default + file override + the
123     runtime mutation that backend/main.py::main performs for --output-dir."""
124     cfg = AppConfig()
125     assert cfg.output.output_dir == "output" # built-in default
126
127     # The CLI override path: main() assigns args.output_dir onto the typed model.
128     cfg.output.output_dir = "/tmp/custom_out"
129     assert cfg.output.output_dir == "/tmp/custom_out"
130
131     # File override is honored on load.
132     with tempfile.TemporaryDirectory() as td:

```

```

133         p = os.path.join(td, "out_override.json")
134         with open(p, "w") as f:
135             json.dump({"output": {"output_dir": "reels", "db_name": "x.db"}}, f)
136         loaded = load_config(p)
137         assert loaded.output.output_dir == "reels"
138         assert loaded.output.db_name == "x.db"
139
140
141     def test_get_returns_dict_for_nested_sections_legacy_compat():
142         """Regression: legacy consumers (validators/segmenters) do
143         ``config.get("validation", {}).get("min_resolution_width", ...)``. The dict-compat
144         .get must return nested SECTIONS as plain dicts (not Pydantic models), or that
145         chained access raises 'X object has no attribute get' on every real run."""
146         cfg = AppConfig()
147         validation = cfg.get("validation", {})
148         assert isinstance(validation, dict)
149         assert validation.get("min_resolution_width") == 1280
150         # nested-of-nested is also dict-accessible
151         assert isinstance(validation.get("relevance"), dict)
152         assert validation["relevance"].get("court_pixels_min_ratio") == 0.05
153         assert cfg.get("indexing", {}).get("default_segmenter") == "yolo_hybrid"
154         # scalar leaves are returned unchanged; a present scalar ignores the default arg;
155         # a missing key falls back to the default
156         assert cfg.get("ai_provider") == "gemini"
157         assert cfg.get("ai_provider", "ignored-default") == "gemini"
158         assert cfg.get("does_not_exist", "fallback") == "fallback"
159         # leaf keys inside any section resolve via the dynamic section scan (not a hardcoded
160         # list), so a future top-level section's keys would be discoverable too
161         assert cfg.get("min_resolution_width") == 1280 # lives inside `validation`
162         assert cfg.get("begin_padding") == 0.5 # lives inside `stitching`
163

```

15. user (2026-06-13T18:45:19.887Z)

```

1     """WASB shuttle-trajectory runner (WSL2 subprocess adapter).
2
3     Windows-side counterpart to backend/detectors/wasb_infer.py. It:
4     1. stages the video into the WSL filesystem (fast local reads, not /mnt),
5     2. copies wasb_infer.py into the WASB repo's src/ dir,
6     3. runs it in the WSL `wasb` conda env -> a Frame,Visibility,X,Y trajectory CSV,
7     4. reuses the (model-agnostic) trajectory->action-window logic from tracknet_runner.
8
9     So the only WASB-specific bits live here; the windowing + CSV parsing are shared
10    with the TrackNet path. Verified inference core: wasb_infer.py (see its docstring).
11    """
12    import os
13    import shlex
14    import logging
15    import subprocess
16    from dataclasses import dataclass
17    from typing import Dict, Any, Optional, Tuple
18
19    # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical.
20    # parse_trajectory_csv / trajectory_to_action_windows are @staticmethods on
21    TrackNetRunner.
22    from backend.pipeline.detectors.tracknet_runner import to_wsl_mnt_path, TrackNetRunner
23    from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
24    wsl_tilde_quote
25
26    logger = logging.getLogger(__name__)
27
28    # Windows path to the inference wrapper we stage into WSL.
29    _INFER_WIN = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")

```

```

30     @dataclass
31     class WasbConfig:
32         """Resolved settings for a WASB WSL run (built from config.json ->
indexing.wasb)."""
33         distro: str = "Ubuntu"
34         repo_dir: str = "~/models/WASB-SBDT"
35         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36         conda_env: str = "wasb"
37         weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
38         sport: str = "badminton"
39         wsl_stage_dir: str = "~/clips"
40         timeout_sec: int = 1800 # 30 min; a hung GPU/conda call is killed (0 = no timeout)
41         keep_frames: bool = False # retain staged video + frame cache after success (debug
only)
42         min_free_gb: float = 0.0 # warn if WSL free space below this before a run (0 =
off)
43
44     @classmethod
45     def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "WasbConfig":
46         w = (idx_cfg or {}).get("wasb", {}) or {}
47         return cls(
48             distro=w.get("wsl_distro", "Ubuntu"),
49             repo_dir=w.get("repo_dir", "~/models/WASB-SBDT"),
50             conda_sh=w.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
51             conda_env=w.get("conda_env", "wasb"),
52             weights_path=w.get("weights_path",
53                               "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"),
54             sport=w.get("sport", "badminton"),
55             wsl_stage_dir=w.get("wsl_stage_dir", "~/clips"),
56             timeout_sec=int(w.get("timeout_sec", 1800)),
57             keep_frames=bool(w.get("keep_frames", False)),
58             min_free_gb=float(w.get("min_free_gb", 0.0)),
59         )
60
61
62     class WasbRunner(WslCommandMixin, DetectorRunner):
63         def __init__(self, cfg: WasbConfig):
64             self.cfg = cfg
65
66         def healthcheck(self) -> Tuple[bool, str]:
67             """Verify the WSL `wasb` env imports torch and sees a GPU. Returns (ok, msg)."""
68             cmd = (f"conda activate {self.cfg.conda_env} && "
69                  f"python -c \"import torch; print('torch', torch.__version__, "
70                  f"'cuda', torch.cuda.is_available())\"")
71             try:
72                 res = self._wsl_bash(cmd)
73             except FileNotFoundError:
74                 return False, "`wsl` not found ? Windows + WSL2 required."
75             except subprocess.TimeoutExpired:
76                 return False, "WSL healthcheck timed out."
77             out = (res.stdout or "").strip()
78             if res.returncode != 0:
79                 return False, f"WSL `wasb` env check failed: {(res.stderr or out).strip()}"
80             return True, out
81
82         def _stage_video(self, video_win_path: str, dest_name: str) -> Optional[str]:
83             src_mnt = to_wsl_mnt_path(video_win_path)
84             dest = f"{self.cfg.wsl_stage_dir}/{dest_name}"
85             cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}
{wsl_tilde_quote(dest)} && echo OK"
86             try:
87                 res = self._wsl_bash(cmd)
88             except subprocess.TimeoutExpired:

```

```

89         logger.error("Staging video into WSL timed out.")
90         return None
91     if res.returncode != 0 or "OK" not in (res.stdout or ""):
92         logger.error("Failed to stage video into WSL: %s", (res.stderr or
res.stdout).strip())
93         return None
94     return dest
95
96     def _stage_infer_script(self) -> bool:
97         """Copy wasb_infer.py into the WASB repo src/ so it can import WASB modules."""
98         src_mnt = to_wsl_mnt_path(_INFER_WIN)
99         cmd = f"cp {shlex.quote(src_mnt)} {self.cfg.repo_dir}/src/wasb_infer.py && echo
OK"
100        res = self._wsl_bash(cmd)

```

16. assistant (2026-06-13T18:45:21.647Z)

Now let me check how the config is used in the main pipeline and look at any other usage patterns:

17. user (2026-06-13T18:45:23.063Z)

```

backend\main.py:598:         threshold = cfg.get("indexing", {}).get("motion_threshold",
0.08) # type: ignore[attr-defined]
backend\config\models.py:57:     Mirrors
backend\pipeline\detectors\wasb_runner.py::WasbConfig.from_indexing_cfg so
backend\pipeline\detectors\person_detector.py:71:         idx_cfg = self.config.get("indexing",
{})
backend\pipeline\detectors\person_detector.py:172:         score_thresh =
self.config.get("indexing", {}).get("detector_score_threshold", 0.5)
backend\pipeline\detectors\person_detector.py:251:         min_window_duration =
self.config.get("indexing", {}).get("min_action_window_duration", 2.0)
backend\reporting\harvest.py:216:     config_default = (config.get("indexing") or
{}).get("default_segmenter")
backend\pipeline\segmenters\gemini.py:38:         idx_cfg = self.config.get("indexing", {})
backend\pipeline\segmenters\gemini.py:60:         debug_duration = self.config.get("indexing",
{}).get("debug_duration")
backend\pipeline\segmenters\gemini.py:110:         idx_cfg = self.config.get("indexing", {})
backend\pipeline\segmenters\gemini.py:199:         max_workers = max(1,
int(self.config.get("indexing", {}).get("ai_max_workers", 5)))
backend\pipeline\segmenters\gemini.py:327:         idx_cfg = self.config.get("indexing", {})
backend\pipeline\detectors\tracknet_runner.py:46:     def from_indexing_cfg(cls, idx_cfg:
Dict[str, Any]) -> "TrackNetConfig":
backend\pipeline\segmenters\tracknet_hybrid.py:35:         cfg =
TrackNetConfig.from_indexing_cfg(idx_cfg)
backend\pipeline\detectors\wasb_runner.py:45:     def from_indexing_cfg(cls, idx_cfg: Dict[str,
Any]) -> "WasbConfig":
backend\pipeline\segmenters\motion.py:41:         idx_cfg = self.config.get("indexing", {})
backend\pipeline\segmenters\yolo_hybrid.py:69:         idx_cfg = self.config.get("indexing", {})
backend\pipeline\segmenters\wasb_hybrid.py:37:         cfg =
WasbConfig.from_indexing_cfg(idx_cfg)
backend\pipeline\segmenters\trajectory_hybrid.py:96:         idx_cfg =
self.config.get("indexing", {})

```

18. assistant (2026-06-13T18:45:24.335Z)

Perfect. Now let me check one more file to understand the pattern for TrackNetConfig:

19. user (2026-06-13T18:45:27.130Z)

```

1     """TrackNet shuttle-trajectory runner (WSL2 subprocess adapter).
2
3     This bridges the Windows-side indexer to a TensorFlow TrackNetV4 install living
4     inside WSL2 (ADR-001 in docs/DECISIONS.md). The flow is:

```

```

5
6     1. Translate the Windows video path to its WSL-visible form (and optionally
7       stage it into the WSL filesystem to avoid slow /mnt/c per-frame reads).
8     2. Invoke ``src/predict.py`` inside the TrackNetV4 conda env via ``wsl``.
9     3. Read back the per-frame ``Frame,Visibility,X,Y`` CSV it writes.
10    4. Convert that trajectory into high-motion "action windows" ? the same
11       candidate-rally shape HybridYoloSegmenter produces ? so the rest of the
12       pipeline (AI handoff, DB population) is unchanged.
13
14    Nothing here imports TensorFlow; all model code stays in WSL.
15    ""
16
17    import os
18    import csv
19    import shlex
20    import logging
21    import subprocess
22    from dataclasses import dataclass
23    from typing import List, Dict, Any, Optional, Tuple
24
25    from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote
26
27    logger = logging.getLogger(__name__)
28
29
30    @dataclass
31    class TrackNetConfig:
32        """Resolved settings for a TrackNet WSL run (built from config.json ->
indexing.tracknet)."""
33        distro: str = "Ubuntu"
34        repo_dir: str = "~/models/TrackNetV4"          # WSL path to the cloned repo
35        conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36        conda_env: str = "TrackNetV4"
37        weights_path: str = ""                        # WSL path to the .h5/.keras weights
38        (REQUIRED)
39        queue_length: int = 5
40        stage_in_wsl: bool = True                     # copy video into WSL fs first
41        (perf)
42        wsl_stage_dir: str = "~/clips"                 # where staged videos/outputs live
43        in WSL
44        timeout_sec: int = 1800                        # 30 min; kills a hung call (0 = no
timeout)
45        keep_frames: bool = False                     # retain staged video after success
46        (debug only)
47        min_free_gb: float = 0.0                      # warn if WSL free space below this
(0 = off)
48
49
50    @classmethod
51    def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "TrackNetConfig":
52        tn = (idx_cfg or {}).get("tracknet", {}) or {}
53        return cls(
54            distro=tn.get("wsl_distro", "Ubuntu"),
55            repo_dir=tn.get("repo_dir", "~/models/TrackNetV4"),
56            conda_sh=tn.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
57            conda_env=tn.get("conda_env", "TrackNetV4"),
58            weights_path=tn.get("weights_path", ""),
59            queue_length=int(tn.get("queue_length", 5)),
60            stage_in_wsl=bool(tn.get("stage_in_wsl", True)),
61            wsl_stage_dir=tn.get("wsl_stage_dir", "~/clips"),
62            timeout_sec=int(tn.get("timeout_sec", 1800)),
63            keep_frames=bool(tn.get("keep_frames", False)),
64            min_free_gb=float(tn.get("min_free_gb", 0.0)),
65        )
66

```

```

62
63 def to_wsl_mnt_path(win_path: str) -> str:
64     """Translate a Windows path (C:\\Users\\x) to its WSL /mnt form (/mnt/c/Users/x).
65
66     Already-POSIX paths (starting with / or ~) are returned unchanged so the
67     same helper is safe to call on either kind of path.
68     """
69     p = str(win_path)
70     if p.startswith("/") or p.startswith("~"):
71         return p
72     p = p.replace("\\", "/")
73     if len(p) >= 2 and p[1] == ":":
74         drive = p[0].lower()
75         return f"/mnt/{drive}{p[2:]}"
76     return p
77
78
79 @dataclass
80 class TrajectoryPoint:
81     frame: int
82     visible: bool
83     x: float
84     y: float
85
86
87 class TrackNetRunner(WslCommandMixin, DetectorRunner):
88     """Runs TrackNet predict.py in WSL and turns the output CSV into action windows.
89
90     Implements the common DetectorRunner contract so a future Linux-native or
91     alternative trajectory runner can be substituted without touching the hybrids.
92     """
93
94     def __init__(self, cfg: TrackNetConfig):
95         self.cfg = cfg
96
97     def healthcheck(self) -> Tuple[bool, str]:
98         """Verify the WSL env is usable: conda env exists, TF imports, GPU visible.
99
100         Returns (ok, message). Cheap to call before a long run.

```

20. assistant (2026-06-13T18:45:28.456Z)

Now let me verify model validation setup and check if there are any special handling for nested configs:

21. user (2026-06-13T18:45:29.575Z)

```

backend\\config\\models.py:6:The models support graceful migration from old config.json files via
extra="allow".
backend\\config\\models.py:59: because nested config sections defaulted to extra='ignore'.
backend\\config\\models.py:114: # Here we just allow any string for forward compatibility.
backend\\config\\models.py:166: allowed_video_root: Optional[str] = None
backend\\config\\models.py:170: # arrive with PR-3 identity scoping. ? In hosted mode, set
allowed_video_root to
backend\\config\\models.py:177: allowed_upload_extensions: List[str] = ["mp4", "mov"]
backend\\config\\models.py:200: model_config = {
backend\\config\\models.py:201:     "extra": "allow",
backend\\config\\loader.py:29: kept by AppConfig's ``extra="allow"`` but never read by typed
code, and a
backend\\config\\loader.py:61: Unknown keys in the file are tolerated (extra="allow" on the
model).

```

22. assistant (2026-06-13T18:45:30.952Z)

Perfect. Now I have all the information needed. Let me compile the structured report:

23. user (2026-06-13T18:45:55.565Z)

Structured output provided successfully

24. assistant (2026-06-13T18:46:00.775Z)

Done. I've mapped the typed config structure for the badminton-highlight-indexer and provided exact additions needed.

Key findings:

1. **IndexingConfig** is a Pydantic model (line 78) with scalar fields and two nested sub-configs (`tracknet`, `wasb`) declared as `Field(default_factory=...)`.
2. **Nested sub-config pattern:** `TrackNetConfig` (lines 33-52) and `WasbIndexConfig` (lines 54-76) are also Pydantic BaseModels. The comment at 55-59 explicitly warns that these were previously silently dropped because of default `extra='ignore'` on nested sections.
3. **Top-level AppConfig** has `extra="allow"` (line 201) ? unknown top-level keys are kept but unread. Unknown section keys are silently dropped unless caught by the `_unknown_key_paths()` loader warning (loader.py:25-42).
4. **For your additions:**
 - `min_crossings: int = 0` goes directly in `IndexingConfig` after line 94.
 - **FusionConfig** as a new Pydantic class (with `court_polygon: Optional[List[List[float]]]`, `net_axis: Literal["x", "y"]`, `net_position: float`, `expected_players: int`) inserted after `WasbIndexConfig`, then added to `IndexingConfig` as `fusion: FusionConfig = Field(default_factory=FusionConfig)` after the `wasb` field (line 108).
5. **Risk:** Existing dataclass adapters (`WasbConfig.from_indexing_cfg`, `TrackNetConfig.from_indexing_cfg`) won't see these new typed defaults if called on raw dicts ? they use `.get()` with fallbacks. If fusion geometry should feed those runners, their adapters need updating.